IMAGE-TRANSFORMATIONS

Aim

To perform image transformation such as Translation, Scaling, Shearing, Reflection, Rotation and Cropping using OpenCV and Python.

Software Required:

Anaconda - Python 3.7

Algorithm:

Step1:

Import necessary libraries such as OpenCV, NumPy, and Matplotlib for image processing and visualization.

Step2:

Read the input image using cv2.imread() and store it in a variable for further processing.

Step3:

Apply various transformations like translation, scaling, shearing, reflection, rotation, and cropping by defining corresponding functions:

1.Translation moves the image along the x or y-axis.

2.Scaling resizes the image by scaling factors.

3.Shearing distorts the image along one axis.

4.Reflection flips the image horizontally or vertically.

5.Rotation rotates the image by a given angle.

Step4:

Display the transformed images using Matplotlib for visualization. Convert the BGR image to RGB format to ensure proper color representation.

Step5:

Save or display the final transformed images for analysis and use plt.show() to display them inline in Jupyter or compatible environments.

Program:

Developed By:ALAN ZION H

Register Number: 212223240004

i)Original image

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```
image=cv2.imread("imagesss.jpg")
image.shape
```

(168, 300, 3)

```
#Display the images:
plt.imshow(image[:,:,::-1])
plt.title("Original Image")
plt.show()
```

ii)Image Translation

```
# Image Translation:
tx,ty=100,200
M_translation=np.float32([[1,0,tx],[0,1,ty]])
translated_image=cv2.warpAffine(image,M_translation, (673,419))
```

```
plt.imshow(translated_image[:,:,::-1])
plt.title("Translated Image")
plt.axis("on")
plt.show()
```

iii) Image Scaling

```
#scaled image
fx, fy = 5.0, 2.0
scaled_image = cv2.resize(image, None, fx=fx, fy=fy, interpolation=cv2.INTER_LINEAR)
plt.imshow(cv2.cvtColor(scaled_image, cv2.COLOR_BGR2RGB))
```

```
plt.title("Scaled Image")
plt.axis('off')
```

iv)Image shearing

```
shear_matrix = np.float32([[1, 0.5, 0], [0.5, 1, 0]])
sheared_image = cv2.warpAffine(image, shear_matrix, (image.shape[1], image.shape[0]))
plt.imshow(cv2.cvtColor(sheared_image, cv2.COLOR_BGR2RGB))
plt.title("Sheared Image")
plt.axis('off')
```

v)Image Reflection

```
#Reflected image
reflected_image = cv2.flip(image, 2)
plt.imshow(cv2.cvtColor(reflected_image, cv2.COLOR_BGR2RGB))
plt.title("Reflected Image")
plt.axis('off')
```

vi)Image Rotation

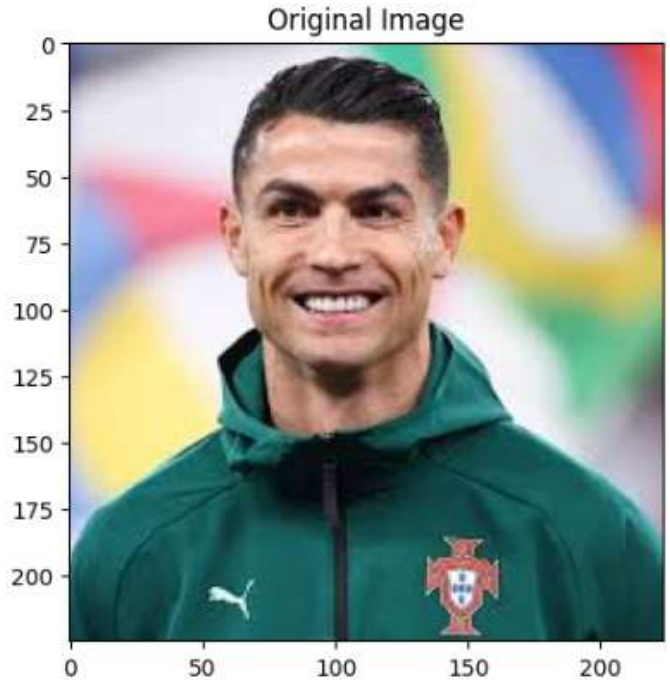
```
# Rotated Image
(height, width) = image.shape[:2]
angle = 60
center = (width // 2, height // 2)
M_rotation = cv2.getRotationMatrix2D(center, angle, 1)
rotated_image = cv2.warpAffine(image, M_rotation, (width, height))
plt.imshow(cv2.cvtColor(rotated_image, cv2.COLOR_BGR2RGB))
plt.title("Rotated Image")
plt.axis('off')
```

vii)Image Cropping

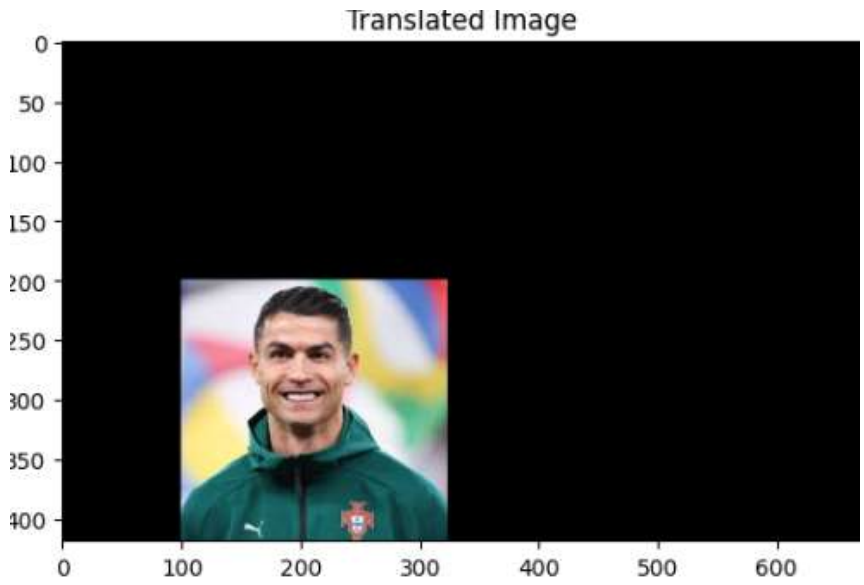
```
# cropped image
x, y, w, h = 100, 100, 200, 150
cropped_image = image[y:y+h, x:x+w]
plt.imshow(cv2.cvtColor(cropped_image, cv2.COLOR_BGR2RGB))
plt.title("Cropped Image")
plt.axis('off')
```

Output:

i)Original image



ii)Image Translation



iii) Image Scaling

Scaled Image

iv)Image shearing

Sheared Image

v)Image Reflection

Reflected Image

vi)Image Rotation

Rotated Image

Releases

No releases published
Create a new release

Packages

No packages published
Publish your first package

Languages

- **Python** 100.0%

Suggested workflows

Based on your tech stack

Django

Build and Test a Django Project

Pylint

Lint a Python application with pylint.

SLSA Generic generator

Generate SLSA3 provenance for your existing release workflows

More workflows